

Member ID: _____

Time: _____

Rank: _____



JAVA PROGRAMMING

(340)

NATIONAL 2022

APPLICATION KNOWLEDGE:

Tollway Customer Database National _____ (420 points)

TOTAL POINTS _____ (420 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The text file imported into the driver program will be error free and all of the data values will be in the correct order. However, you will need to format any numerical values into US currency.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must use Javadoc comments.

Reading Delimited File into Records, and Cloning

You will be creating an entire program that will create customer records that are read from a text document and then create clones of these objects. The customer objects must be cloned because they consist of being made entirely of objects, and cloning is an important time saving process to duplicate these objects in their entirety. If not, the code would have to have subroutines that went through every field of every object nested in the customer object. To clone objects you will be required to use the Cloneable interface for all instantiated objects that will be cloned. Once the objects are cloned, you will be required to write these new cloned objects to their own text document with the amended string flags that allows one to quickly see if the object was cloned. Seeing how some of the skills to complete this challenge are skills you already demonstrated at the state level (reading text files) you will be required to complete this code as well. The program is all automated and requires zero manual input from the user. The results should be instantaneously. There will be little guidance on how to accomplish this task other than the commented sections contained in the source code starter file.

Input:

The program will read the data fields from the “Names.txt” that uses commas as the delimiter. There are only 12 total records to be read, each one having seven fields each. Below is an example of a record without and with actual data:

Without Data

FirstName, LastName, Make, Model, LicensePlate, Deposit, CreditCard#

With Data

Jose, Montana, Ford, F150, ABC-123, 65.00, WHJFHJFO483897

Isaac, Newton, Chevy, Silverado, DFR-345, 89.01, WH24223442897

Output 1: Missing Text File

The first output will print if the “Names.txt” file is missing:

The file doesn't exist.

Output 2: Imported Records

The first output should look similar to the example below. Notice that the top of the record contains its memory address...this needs to be supplied for each record. The memory address will be different for every record and your address will not match the one below (they are proprietary to each machine). Also, take notice that the deposit amount has been formatted into US currency. There will be a total of 12 records for Output 1.

Customer@12edcd21
Name: Jose Montana
Car Information: Ford F150 ABC-123
Payment: \$65.00 WHJFHJFO483897

Customer@7eda2dbb

Name: Isaac Newton

Car Information: Chevy Silverado DFR-345

Payment: \$89.01 WH24223442897

Output 3: Cloned Records

The transitions between Output 1 and Output 2 will be instantaneous. The second set of output will be the cloned objects. To help quickly identify the cloned objects it is required to amend the term "DUPLICATES" under the the memory address, as well as place the string "CLONED" in front of each section of the customer record. There will be a total of 12 cloned records printed. NOTE: placing the string "*****CLONES*****" in between the original records and cloned records is not required.

*****CLONES*****
Customer@1a6c5a9e
DUPLICATES
CLONED Name: Jose Montana
CLONED Car Information: Ford F150 ABC-123
CLONED Payment: \$65.00 WHJFHJFO483897

Customer@37bba400
DUPLICATES
CLONED Name: Isaac Newton
CLONED Car Information: Chevy Silverado DFR-345
CLONED Payment: \$89.01 WH24223442897

Output 4: Clones.txt

The final output from the program will entail the cloned records being written to a new text file called “Clones.txt”. The file should have all records formatted in a similar fashion except excluding the memory address of the object. In addition, at the top of the text file, you must also include in the output a single instance of the exact time and date for every time the text file is written. This exact time can be captured from the machine and it will vary based upon when and where the text file is created. There will be a total of 12 records printed below the current date.

```
Mon Nov 23 18:10:17 CST 2020
DUPLICATES
CLONED Name: Jose Montana
CLONED Car Information: Ford F150 ABC-123
CLONED Payment: $65.00 WHJFHJF0483897

DUPLICATES
CLONED Name: Isaac Newton
CLONED Car Information: Chevy Silverado DFR-345
CLONED Payment: $89.01 WH24223442897
```

Requirements:

- 1) You will create an application with the main class named **TollwayCustomerDataBaseNational**
- 2) The **TollwayCustomerDataBaseNational** class will perform the following actions:
 - a) Read the “Names.txt” file and break the information from the text document to be placed in the appropriated object that will be used to create each customer object (this is the same process as the State competition).
 - i) **Customer** is created with **Name**, **CarInfo** and **Payment** objects.
 - (1) **Name** is constructed with the FirstName, and LastName.
 - (2) **CarInfo** is constructed with the Make, Model, and LicensePlate.
 - (3) **Payment** is constructed with the Deposit and CreditCard# fields.
 - ii) Remember: All of the customer objects should be stored in a List.
 - iii) You must use “try... catch” for exception handling in TollwayCustomerDataBaseState class when reading (FileNotFoundException), inputting (IOException) and writing (IOException) files.
 - b) Clone the customer objects and store them into their own List.
 - i) NOTE: the clone method will be a public method in the **Customer**, **Name**, **CarInfo**, and **Payment** classes. If properly created, you should only have to call on the customer clone method.
 - c) Print out the original records in the required format, then print out the cloned records in the required format.
 - d) Write the cloned records in the required format to a text document called “Clones.txt” that includes the current date and time per the computer writing the text file.

- i) Date Hint: `java.util.Date date = new java.util.Date( );`
- 3) **Customer**, **Name**, **CarInfo** and **Payment** classes:
 - a) You will need to properly create the variables and constructor for each one of these classes.
 - b) All four must use the **Cloneable** and **InformationRetrieval** interfaces.
 - i) **Cloneable** interface (source code is not provided since this is part of the `java.util.package`; NOTE: the package has already been imported for you in the student starter file):
 - (1) The only required method that needs to be used for this interface: `Object clone( )`.
 - (2) Hint: the method returns an `Object`...you will need to cast this into a the appropriate object you want to return.
 - (3) Hint: call `super.clone( )` to make a clone of current object.
 - (4) Hint: call `super.clone( )` in a `try/catch` statement that catches the `CloneNotSupportedException` to catch an error if you try to clone in an object that does not properly implement the interface.
 - (5) Hint: you will need to call `super.clone( )` on all objects being cloned; including any nested objects.
 - ii) **InformationRetrieval** interface is not very complex and only consists of two methods that must be used (this interface source code is available in the student starter file):
 - (1) `String getInfo( )`: write this method to return the formatted string for all output. Each class needs to have its own implementation of this method.
 - (2) `String confirmed( )`: write this method to return the string "CLONED" that will be used to amended to the cloned output per the formatting requirements.
- 4) The program will display the output like the examples above for all records with the proper formatting.

Your application will be graded on the following criteria:		
Solution and Project		
The project is present on the flash drive		10 points
The projects main class is named TollwayCustomerDataBaseNational	_____	10 points
The projects helper class is named Customer	_____	10 points
The projects helper class is named Name	_____	10 points
The projects helper class is named CarInfo	_____	10 points
The projects helper class is named Payment	_____	10 points
The interface is called InformationRetrieval	_____	10 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
If the program does <i>not</i> execute, then the remaining items in this section receive a score of zero.		
The program displays in the console if the “Names.txt” file being read is not found: “The file doesn't exist.”	_____	10 points
The program displays the entire set of records imported from “Names.txt” in the proper formatting per the instructions (12 total records)	_____	10 points
The program clones and displays the cloned records in the proper formatting per the instructions (12 total records)	_____	30 points
Program writes only the cloned records to a text file in the proper format; file must be called “Clones.txt”	_____	40 points
The “Clones.txt” contains the current date and time at the top of the document for every time it is written	_____	20 points
Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present	_____	10 points
Methods and code sections are commented	_____	20 points
Helper Classes each implement InformationRetrieval interface (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each implement Cloneable interface (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each use InformationRetrieval abstract method confirmed(); properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each use InformationRetrieval abstract method getInfo(); properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points

Helper Classes each use Cloneable abstract method clone(); properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Customer objects read from “Names.txt” are stored in a List	_____	10 points
Cloned customer objects are stored in a separate List	_____	10 points
Code uses try... catch for FileNotFoundException exception handling in TollwayCustomerDataBaseState class when reading the file; commenting must be used to identify.	_____	20 points
Code uses try... catch for IOException exception handling in TollwayCustomerDataBaseState class when inputting from the file; commenting must be used to identify.	_____	20 points
Code uses try... catch for IOException exception handling in TollwayCustomerDataBaseState class when writing to a file; commenting must be used to identify.	_____	20 points
Object clone(): Code uses try... catch for exception handling in all helper class implementing Cloneable interface (catches CloneNotSupportedException) 5 points each	_____	20 points

Total Points = _____ / 420 points